

Applied Type Theory

Лекция 6: System F ($\lambda 2$) — Полиморфизм второго порядка

Воронов Михаил Сергеевич

ВМК МГУ

Осень 2025

- 1 Введение в полиморфизм
- 2 Типовые переменные и контекст
- 3 Правила типизации System F
- 4 Стили типизации и разрешимость задач
- 5 Карри-стиль с явной полиморфной нотацией ($\lambda/[]$ как сахар)
- 6 Parametricity и «бесплатные теоремы»
- 7 Hindley-Milner vs System F: детальное сравнение
- 8 Дополнительные темы: кодирование и ограничения
- 9 Заключение

План лекции

- 1 Введение в полиморфизм
- 2 Типовые переменные и контекст
- 3 Правила типизации System F
- 4 Стили типизации и разрешимость задач
- 5 Карри-стиль с явной полиморфной нотацией ($\lambda/[]$ как сахар)
- 6 Parametricity и «бесплатные теоремы»
- 7 Hindley-Milner vs System F: детальное сравнение
- 8 Дополнительные темы: кодирование и ограничения
- 9 Заключение

Проблема: дублирование кода для разных типов

В простом типизированном λ -исчислении каждая функция имеет конкретный тип:

- $\lambda x^\alpha. x : \alpha \rightarrow \alpha$ — функция тождества для типа α
- $\lambda x^\beta. x : \beta \rightarrow \beta$ — функция тождества для типа β
- $\lambda x^{(\beta \rightarrow \beta)}. x : (\beta \rightarrow \beta) \rightarrow (\beta \rightarrow \beta)$ — функция тождества для типа $\beta \rightarrow \beta$

Проблема

Одна и та же функция (например, тождественная функция) должна быть написана *отдельно* для каждого типа.

Мы не можем написать *универсальную* функцию тождества, которая работает для *любого* типа.

Вопрос: можно ли написать функцию, которая работает для **всех** типов одновременно?

Решение: полиморфизм типов (System F)

Идея: добавить в язык возможность абстрагироваться не только по значениям, но и по *типам*.

Полиморфная функция тождества (Church-style)

$$\Lambda\alpha. \lambda x^\alpha. x : \forall\alpha. \alpha \rightarrow \alpha$$

Эта функция работает для **любого** типа α .

Примеры применения (в стиле Карри для краткости):

- Применим к числу: $(\lambda x. x) 5 : \text{Nat}$
- Применим к булеву значению: $(\lambda x. x) \text{true} : \text{Bool}$
- Применим к функции: $(\lambda x. x) (\lambda y. y) : (\beta \rightarrow \beta)$

Полиморфная константная функция

$$\lambda x. \lambda y. x : \forall\alpha. \forall\beta. \alpha \rightarrow \beta \rightarrow \alpha$$

Функция, которая возвращает первый аргумент независимо от типов.

System F: синтаксис полиморфных типов

System F (также известная как полиморфное λ -исчисление второго порядка) расширяет STLC квантором всеобщности $\forall$.

Синтаксис типов

$$\tau ::= \alpha \mid \tau \rightarrow \tau \mid \forall \alpha. \tau$$

где:

- α — типовая переменная
- $\tau \rightarrow \tau$ — тип функции (как раньше)
- $\forall \alpha. \tau$ — **пoлимoрфный тип** (квантор всеобщности)

Примеры полиморфных типов:

- $\forall \alpha. \alpha \rightarrow \alpha$ — тип функции тождества
- $\forall \alpha. \forall \beta. \alpha \rightarrow \beta \rightarrow \alpha$ — тип константной функции
- $\forall \alpha. \alpha \rightarrow (\forall \beta. \forall \gamma. \beta \rightarrow \gamma) \rightarrow \alpha$ — более сложный тип

Сравнение STLC и System F

Аспект	STLC ($\lambda_{\rightarrow}$)	System F (λ_2)
Типы	$\tau ::= \alpha \mid \tau \rightarrow \tau$	$\tau ::= \alpha \mid \tau \rightarrow \tau \mid \forall \alpha. \tau$
Термы	$M ::= x \mid \lambda x. M \mid MN$	$M ::= \dots \mid \Lambda \alpha. M \mid M[\tau]$
Абстракция	по термам	по термам И типам
Уровни	1 (только термы)	2 (термы + типы)
Порядок	первый	второй
Полиморфизм	нет	есть ($\forall$)
Примеры типов	$\lambda x:\text{Nat}. x + 1$ $\text{Nat} \rightarrow \text{Nat}$	$\Lambda \alpha. \lambda x:\alpha. x$ $\forall \alpha. \alpha \rightarrow \alpha$

Ключевое отличие: System F добавляет абстракцию по типам ($\Lambda \alpha$) и применение к типам ($M[\tau]$).

Упрощённый синтаксис: мономорфные типы

Иногда полезно рассматривать *подмножество* типов без кванторов:

Мономорфные типы τ_0

$$\tau_0 ::= \alpha \mid \tau_0 \rightarrow \tau_0$$

Это типы без квантора $\forall$ — обычные типы из STLC.

Полные типы τ

$$\tau ::= \forall \alpha. \tau_0 \mid \tau_0$$

Полиморфный тип — это либо квантор всеобщности над мономорфным типом, либо просто мономорфный тип.

Интуиция: квантор $\forall$ находится «снаружи», а внутри — обычные стрелочные типы.

Примечание: это упрощённая грамматика для учебных целей. В полной System F кванторы могут быть вложенными.

План лекции

- 1 Введение в полиморфизм
- 2 Типовые переменные и контекст**
- 3 Правила типизации System F
- 4 Стили типизации и разрешимость задач
- 5 Карри-стиль с явной полиморфной нотацией ($\lambda/[]$ как сахар)
- 6 Parametricity и «бесплатные теоремы»
- 7 Hindley-Milner vs System F: детальное сравнение
- 8 Дополнительные темы: кодирование и ограничения
- 9 Заключение

Типовая переменная: связанная vs свободная

В полиморфных типах переменные бывают двух видов:

Связанная типовая переменная

В типе $\forall \alpha. \alpha \rightarrow \alpha$ переменная α **связана** квантором $\forall$.

Это аналог связанной переменной в λ -терме: $\lambda x. x$

Свободная типовая переменная

В типе $\alpha \rightarrow \beta$ переменные α и β **свободны**.

Они должны быть определены в контексте (области видимости).

Замечание: когда мы пишем $\alpha \in \mathcal{V}$, это означает, что α — типовая переменная из множества всех типовых переменных.

Контекст: область видимости типов

Контекст Γ — это список объявлений переменных и их типов.

Два вида переменных в контексте

- $\alpha : *$ — **типовая переменная** α имеет вид (kind) $*$
- $x : \alpha$ — **термовая переменная** x имеет тип α

Пример контекста:

$$\Gamma = \{ \alpha : *, x : \alpha \rightarrow \alpha \}$$

Этот контекст говорит:

- Есть типовая переменная α (некий абстрактный тип)
- Есть термовая переменная x типа $\alpha \rightarrow \alpha$ (функция от α к α)

Важно: контекст **упорядочен** — использование переменной возможно только после её объявления.

Что такое kind (вид)?

Определение

Kind (вид) — это «тип типов». Он указывает, к какому классу принадлежит типовое выражение.

В простейшем случае (System F):

$$\alpha : * \iff \llcorner \alpha \text{ является типом} \gg$$

Примеры:

- $\text{Nat} : *$ — натуральные числа это тип
- $\text{Bool} : *$ — булевы значения это тип
- $(\text{Nat} \rightarrow \text{Bool}) : *$ — функциональный тип это тоже тип
- $\alpha : *$ — типовая переменная α представляет некий тип

Интуиция: запись $\alpha \in \mathcal{V}$ означает, что α — это типовая переменная. Запись $\alpha : *$ означает, что эта переменная имеет вид «тип».

Пример: типизация в контексте

Рассмотрим контекст:

$$\Gamma = \{ \alpha : *, x : \alpha \rightarrow \alpha \}$$

Что можно вывести в этом контексте?

Расширим контекст: $\Gamma' = \{ \alpha : *, x : \alpha \rightarrow \alpha, a : \alpha \}$

- $\Gamma' \vdash x : \alpha \rightarrow \alpha$ ✓ (переменная x объявлена в контексте)
- $\Gamma' \vdash \lambda y^\alpha. y : \alpha \rightarrow \alpha$ ✓ (можем использовать α как тип)
- $\Gamma' \vdash x a : \alpha$ ✓ (применяем x к аргументу типа α)

Контрпример: $\Gamma \not\vdash x x$ — нельзя, т.к. x ожидает α , а не $\alpha \rightarrow \alpha$

Ключевой момент: типовая переменная α должна быть объявлена в контексте перед тем, как её можно использовать в типах термов.

Упорядочение в System F: это требование называется «scoping» или «упорядоченный контекст» — типовые переменные нельзя использовать до их объявления.

План лекции

- 1 Введение в полиморфизм
- 2 Типовые переменные и контекст
- 3 Правила типизации System F**
- 4 Стили типизации и разрешимость задач
- 5 Карри-стиль с явной полиморфной нотацией ($\lambda/[]$ как сахар)
- 6 Parametricity и «бесплатные теоремы»
- 7 Hindley-Milner vs System F: детальное сравнение
- 8 Дополнительные темы: кодирование и ограничения
- 9 Заключение

Корректность типов: выводимость из контекста

Прежде чем типизировать термы, нужно убедиться, что сами типы корректны.

Выводимый тип

Тип σ **выводим** из контекста Γ (обозначение: $\Gamma \vdash \sigma : *$), если:

$$FV_{\text{type}}(\sigma) \subseteq \text{dom}_{\text{type}}(\Gamma)$$

То есть все свободные типовые переменные в σ объявлены в типовой части контекста Γ .

Примеры:

- $\{\alpha : *, \beta : *\} \vdash (\alpha \rightarrow \beta) : *$ ✓ (обе переменные в контексте)
- $\{\alpha : *\} \vdash (\alpha \rightarrow \beta) : *$ × (β не объявлена)
- $\emptyset \vdash (\forall \alpha. \alpha \rightarrow \alpha) : *$ ✓ (α связана квантором)

Допустимый контекст

Не любая последовательность объявлений является корректным контекстом.

Правила построения допустимого контекста

Контекст Γ **допустим** (well-formed), если он построен по одному из правил:

(Empty) Пустой контекст допустим:

$$\frac{}{\emptyset \vdash} \text{EMPTY}$$

(Type var) Можно добавить новую типовую переменную:

$$\frac{\Gamma \vdash \quad \alpha \notin \text{dom}(\Gamma)}{\Gamma, \alpha : * \vdash} \text{TVar}$$

(Term var) Можно добавить термовую переменную с корректным типом:

$$\frac{\Gamma \vdash \quad \Gamma \vdash \sigma : * \quad x \notin \text{dom}(\Gamma)}{\Gamma, x : \sigma \vdash} \text{TERMVar}$$

Примеры корректных и некорректных контекстов

Корректные контексты:

- $\emptyset$ — пустой контекст
- $\alpha : *$ — одна типовая переменная
- $\alpha : *, x : \alpha$ — переменная и терм её типа
- $\alpha : *, \beta : *, f : \alpha \rightarrow \beta$ — функция между двумя типами
- $\alpha : *, x : \alpha, y : \alpha$ — два терма одного типа

Некорректные контексты:

- $x : \alpha$ — типовая переменная α не объявлена
- $\alpha : *, \alpha : *$ — дублирование переменной
- $\alpha : *, x : \alpha, x : \alpha$ — дублирование терма

Вывод: контекст должен быть упорядоченным множеством без повторений, где каждая переменная объявлена до использования.

Правила типизации: обзор

Правила типизации System F расширяют STLC двумя новыми правилами для работы с $\forall$.

Все правила типизации

- ❶ **(Var)** — использование переменной из контекста
- ❷ **(Intro $\rightarrow$)** — введение функции (абстракция λ)
- ❸ **(Elim $\rightarrow$)** — применение функции (аппликация)
- ❹ **(Intro $\forall$)** — введение полиморфизма (типовая абстракция)
- ❺ **(Elim $\forall$)** — инстанцирование полиморфного типа (типовая аппликация)

Новинка: правила 4 и 5 позволяют работать с типовыми параметрами.

Ключевая идея: абстракция и аппликация происходят на двух уровнях — уровне термов (как в STLC) и уровне типов (новое в System F).

Правила $\forall$ (Church-style)

В стиле Чёрча типовые абстракция и аппликация **явные** в термах:

$$\frac{\Gamma, \alpha : * \vdash M : \sigma \quad \alpha \notin \text{FV}(\Gamma)}{\Gamma \vdash \Lambda \alpha. M : \forall \alpha. \sigma} \text{INTRO}\forall^{\text{Ch}} \qquad \frac{\Gamma \vdash M : \forall \alpha. \sigma \quad \Gamma \vdash \tau : *}{\Gamma \vdash M[\tau] : \sigma[\alpha := \tau]} \text{ELIM}\forall^{\text{Ch}}$$

Примеры:

- $\Lambda \alpha. \lambda x^\alpha. x : \forall \alpha. \alpha \rightarrow \alpha$ — явная типовая абстракция
- $(\Lambda \alpha. \lambda x^\alpha. x) [\text{Nat}] : \text{Nat} \rightarrow \text{Nat}$ — явная типовая аппликация

Правила $\forall$ (Curry-style)

В стиле Карри типовые абстракция и аппликация **неявные** (терм не меняется):

$$\frac{\Gamma, \alpha : * \vdash M : \sigma \quad \alpha \notin \text{FV}(\Gamma)}{\Gamma \vdash M : \forall \alpha. \sigma} \text{INTRO}\forall^{\text{Cu}} \qquad \frac{\Gamma \vdash M : \forall \alpha. \sigma \quad \Gamma \vdash \tau : *}{\Gamma \vdash M : \sigma[\alpha := \tau]} \text{ELIM}\forall^{\text{Cu}}$$

Примеры:

- $\lambda x. x : \forall \alpha. \alpha \rightarrow \alpha$ — терм без типовых аннотаций
- $\lambda x. x : \text{Nat} \rightarrow \text{Nat}$ — тот же терм, другой тип (инстанциация неявная)

Разница: в Чёрче терм содержит всю информацию, в Карри — типы выводятся.

Правило (Var): использование переменной

Правило переменной

$$\frac{x : \sigma \in \Gamma}{\Gamma \vdash x : \sigma} \text{VAR}$$

Если переменная x объявлена в контексте с типом σ , то терм x имеет тип σ .

Примеры:

- Контекст: $\Gamma = \{\alpha : *, x : \alpha\}$
Вывод: $\Gamma \vdash x : \alpha$ ✓
- Контекст: $\Gamma = \{\alpha : *, \beta : *, f : \alpha \rightarrow \beta\}$
Вывод: $\Gamma \vdash f : \alpha \rightarrow \beta$ ✓

Примечание: это то же правило, что и в STLC — ничего не меняется.

Правило (Intro $\rightarrow$): введение функции

Правило абстракции

$$\frac{\Gamma, x : \sigma \vdash M : \tau}{\Gamma \vdash \lambda x. M : \sigma \rightarrow \tau} \text{INTRO} \rightarrow$$

Если в расширенном контексте $\Gamma, x : \sigma$ терм M имеет тип τ , то λ -абстракция $\lambda x. M$ имеет тип $\sigma \rightarrow \tau$.

Пример: типизация функции тождества

$$\frac{\{\alpha : *, x : \alpha\} \vdash x : \alpha}{\{\alpha : *\} \vdash \lambda x. x : \alpha \rightarrow \alpha}$$

Примечание: это то же правило, что и в STLC. Но теперь σ и τ могут содержать $\forall$.

Правило (Elim $\rightarrow$): применение функции

Правило аппликации

$$\frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ELIM} \rightarrow$$

Если M имеет тип $\sigma \rightarrow \tau$ и N имеет тип σ ,
то применение $M N$ имеет тип τ .

Пример: применение функции к аргументу

$$\frac{\Gamma \vdash f : \alpha \rightarrow \beta \quad \Gamma \vdash x : \alpha}{\Gamma \vdash f x : \beta}$$

где $\Gamma = \{\alpha : *, \beta : *, f : \alpha \rightarrow \beta, x : \alpha\}$

Примечание: это то же правило modus ponens, что и в STLC.

Formation $\forall$: когда тип корректен

Formation (формирование типа)

$$\frac{\Gamma, \alpha : * \vdash \tau : *}{\Gamma \vdash \forall \alpha. \tau : *}$$

Если τ — корректный тип при добавлении α в контекст, то $\forall \alpha. \tau$ — тоже корректный тип.

Интуиция: квантор всеобщности «связывает» типовую переменную, делая тип полиморфным.

Примеры:

- $\emptyset \vdash \forall \alpha. \alpha \rightarrow \alpha : *$ ✓ (тип id)
- $\emptyset \vdash \forall \beta. \beta : *$ ✓ (корректно сформирован, но *необитаем*)
- $\{\alpha : *\} \vdash \forall \beta. \alpha \rightarrow \beta : *$ ✓ (α из контекста)

Introduction $\forall$ (Curry-стиль): обобщение без Λ в терме

Introduction (введение квантора)

$$\frac{\Gamma, \alpha : * \vdash M : \sigma \quad \alpha \notin FV(\Gamma)}{\Gamma \vdash M : \forall \alpha. \sigma} \text{INTRO}\forall$$

Если в расширенном контексте $\Gamma, \alpha : *$ терм M имеет тип σ , то M также имеет полиморфный тип $\forall \alpha. \sigma$ в исходном контексте Γ .

Side condition: α не должна быть свободна в Γ .

Пример: полиморфная функция тождества

$$\frac{\{\alpha : *\} \vdash \lambda x. x : \alpha \rightarrow \alpha}{\emptyset \vdash \lambda x. x : \forall \alpha. \alpha \rightarrow \alpha}$$

Интуиция: если M работает для произвольной типовой переменной α , то M работает для *любого* типа — получаем полиморфизм.

Elimination $\forall$: инстанцирование типа

Elimination (применение к типу)

$$\frac{\Gamma \vdash M : \forall \alpha. \sigma \quad \Gamma \vdash \tau : *}{\Gamma \vdash M : \sigma[\alpha := \tau]} \text{ELIM}\forall$$

Если M имеет полиморфный тип $\forall \alpha. \sigma$ и τ — корректный тип, то M также имеет тип $\sigma[\alpha := \tau]$ (подстановка τ вместо α).

Интуиция: «специализируем» полиморфную функцию под конкретный тип.

Примеры:

- Пусть $id : \forall \alpha. \alpha \rightarrow \alpha$. Тогда:
 - $id[\text{Nat}] : \text{Nat} \rightarrow \text{Nat}$ (специализация к Nat)
 - $id[\text{Bool}] : \text{Bool} \rightarrow \text{Bool}$ (специализация к Bool)
 - $id[\alpha \rightarrow \alpha] : (\alpha \rightarrow \alpha) \rightarrow (\alpha \rightarrow \alpha)$ (специализация к функции)

В стиле Чёрча: применение типа пишется явно: $M[\tau]$.

В стиле Карри/Haskell: применение неявное, выводится автоматически.

Computation: β -редукция на типовом уровне

Computation (β -редукция)

Типовая абстракция, применённая к типу, редуцируется через подстановку:

$$(\Lambda\alpha. M)[\tau] \mapsto M[\alpha := \tau]$$

Аналогично термовому $(\lambda x. M) N \mapsto M[x := N]$, но на уровне типов.

Пример пошаговой редукции:

$$\begin{aligned} & (\Lambda\alpha. \lambda x:\alpha. x)[\text{Nat}] \\ & \mapsto \lambda x:\text{Nat}. x \quad (\text{подставили Nat вместо } \alpha) \\ & : \text{Nat} \rightarrow \text{Nat} \end{aligned}$$

Более сложный пример:

$$\begin{aligned} & (\Lambda\alpha. \Lambda\beta. \lambda x:\alpha. \lambda y:\beta. x)[\text{Nat}][\text{Bool}] \\ & \mapsto (\Lambda\beta. \lambda x:\text{Nat}. \lambda y:\beta. x)[\text{Bool}] \\ & \mapsto \lambda x:\text{Nat}. \lambda y:\text{Bool}. x \end{aligned}$$

Примеры полиморфных типов: константная функция

Задача: вывести тип для $\lambda x. \lambda y. x$

Шаг 1: типизация во внутреннем контексте

$$\{\beta : *, \alpha : *, x : \alpha, y : \beta\} \vdash x : \alpha$$

Шаг 2: абстракция по y

$$\{\beta : *, \alpha : *, x : \alpha\} \vdash \lambda y. x : \beta \rightarrow \alpha$$

Шаг 3: абстракция по x

$$\{\beta : *, \alpha : *\} \vdash \lambda x. \lambda y. x : \alpha \rightarrow \beta \rightarrow \alpha$$

Шаг 4: типовая абстракция по α и β

$$\emptyset \vdash \lambda x. \lambda y. x : \forall \alpha. \forall \beta. \alpha \rightarrow \beta \rightarrow \alpha$$

Результат: универсальная константная функция для любых типов.

Пример: самоприменение полиморфной функции

Вопрос: можно ли применить полиморфную функцию тождества к самой себе?

Рассмотрим контекст: $\Gamma = \{f : \forall \alpha. \alpha \rightarrow \alpha, \beta : *\}$

Построение типа для $(f f)$:

❶ По правилу Var: $\Gamma \vdash f : \forall \alpha. \alpha \rightarrow \alpha$

❷ По правилу Elim $\forall$ с $\tau = \beta \rightarrow \beta$:

$$\Gamma \vdash f : (\beta \rightarrow \beta) \rightarrow (\beta \rightarrow \beta)$$

❸ Снова Elim $\forall$ для второго f с $\tau = \beta$:

$$\Gamma \vdash f : \beta \rightarrow \beta$$

❹ По правилу Elim $\rightarrow$:

$$\Gamma \vdash (f f) : \beta \rightarrow \beta$$

Пример (продолжение): обобщение результата

Мы получили: $\{f : \forall \alpha. \alpha \rightarrow \alpha, \beta : *\} \vdash (f f) : \beta \rightarrow \beta$

Обобщение:

- 1 Применим $\text{Intro}\forall$ по β :

$$\{f : \forall \alpha. \alpha \rightarrow \alpha\} \vdash (f f) : \forall \beta. \beta \rightarrow \beta$$

- 2 Применим $\text{Intro}\rightarrow$ по f :

$$\emptyset \vdash \lambda f. (f f) : (\forall \alpha. \alpha \rightarrow \alpha) \rightarrow (\forall \beta. \beta \rightarrow \beta)$$

Результат

Функция $\lambda f. (f f)$ принимает полиморфную функцию тождества и применяет её к самой себе, возвращая снова полиморфную функцию тождества.

Замечание: в простом STLC самоприменение невозможно, но в System F с полиморфизмом — возможно.

Почему не противоречит запрету $(x x)$ в STLC? Здесь «самоприменение» идёт *через типовую специализацию* (f применяется с разными типовыми аргументами), а не через термовое равенство типов.

Типовые ошибки и контрпримеры

Что НЕЛЬЗЯ в System F:

❶ **Ошибка:** $\Lambda\alpha. \alpha$

Проблема: смешение уровней. α — типовая переменная, а тело $\Lambda\alpha. \cdot$ должно быть *термом*.
Корректно: $\Lambda\alpha. \lambda x^\alpha. x$.

❷ **Ошибка:** $\Lambda\alpha. \lambda x:\beta. x$

Проблема: β не в scope (свободная типовая переменная).

❸ **Ошибка:** $(\Lambda\alpha. \lambda x:\alpha. x) 5$

Проблема: type/term mismatch — применяем типовую функцию к терму, нужно $(\dots)[\text{Nat}] 5$.

❹ **Ошибка:** $\lambda x. (x x)$

Проблема: как в STLC, самоприменение запрещено (occurs check: нужно $\alpha = \alpha \rightarrow \beta$).

Вывод: System F запрещает самоприменение на термовом уровне, но разрешает «самоприменение» полиморфных функций (пример: $f f$ для $f : \forall\alpha. \alpha \rightarrow \alpha$).

Различие правил в упрощённой и полной System F

В наших заметках упоминаются разные варианты правил для prenex и полной System F.

Упрощённая System F ($\lambda_2(\tau_0)$)

В правилах Intro $\rightarrow$ и Elim $\forall$:

- Тип результата и инстанцируемый тип должны быть *мономорфными* ($\in \tau_0$)
- Кванторы $\forall$ только «снаружи»

Полная System F ($\lambda_2(\tau)$)

В правилах Intro $\rightarrow$ и Elim $\forall$:

- Типы могут быть *любыми* ($\in \tau$)
- Кванторы $\forall$ могут быть вложенными

Вывод: упрощённая версия проще для изучения, полная версия более выразительна.

Примечание: в полной System F можно иметь типы вида $\alpha \rightarrow (\forall \beta. \beta \rightarrow \beta)$, что невозможно в упрощённой версии (prenex polymorphism).

План лекции

- 1 Введение в полиморфизм
- 2 Типовые переменные и контекст
- 3 Правила типизации System F
- 4 Стили типизации и разрешимость задач**
- 5 Карри-стиль с явной полиморфной нотацией ($\lambda/[]$ как сахар)
- 6 Parametricity и «бесплатные теоремы»
- 7 Hindley-Milner vs System F: детальное сравнение
- 8 Дополнительные темы: кодирование и ограничения
- 9 Заключение

Два стиля типизации: Чёрч vs Карри

Существуют два подхода к типизации λ -исчисления:

Стиль Чёрча (Church-style, explicit typing)

Типы **явно указаны** в термах:

$$\lambda x^\sigma. M \quad \Lambda \alpha. M$$

- Тип аргумента σ записан в абстракции
- Типовая абстракция $\Lambda \alpha$ явно указывает параметр типа
- Это то, что мы изучали до сих пор

Стиль Карри (Curry-style, implicit typing)

Типы **не указаны** в термах:

$$\lambda x. M \quad M$$

- Термы как в нетипизированном λ -исчислении
- Типы выводятся автоматически

Пример: самоприменение в разных стилях

Рассмотрим контекст: $\Gamma = \{f : \forall \alpha. \alpha \rightarrow \alpha, \beta : *\}$

Задача: типизировать терм $(f f)$

Стиль Чёрча

Типовые аппликации явные:

$$(f [\beta \rightarrow \beta]) (f [\beta])$$

Здесь $[\tau]$ — явное применение типа.

Тип: $\beta \rightarrow \beta$

Проверка типов: простая — все типы уже указаны.

Стиль Карри

Типовые аппликации неявные:

$$(f f)$$

Система должна *вывести*, что нужно применить f к типам $\beta \rightarrow \beta$ и β .

Тип: $\beta \rightarrow \beta$

Вывод типов: сложнее — нужен алгоритм.

Варианты System F

Мы изучали несколько вариантов System F:

$\lambda_2(\tau_0)$ — упрощённая System F

- Кванторы $\forall$ только снаружи
- В Intro $\rightarrow$ и Elim $\forall$ типы мономорфные
- Проще для изучения

$\lambda_2(\tau)$ — полная System F

- Кванторы $\forall$ могут быть вложенными
- Типы вида $\alpha \rightarrow (\forall\beta. \beta \rightarrow \beta)$ допустимы
- Более выразительная система

Стиль типизации:

- **Чёрч** — типы в термах: $\lambda x^\sigma. M$
- **Карри** — типы отдельно: $\lambda x. M$

Разрешимость задач в различных системах

Задача	$\lambda_2(\tau_0)$ (rank-1)		$\lambda_2(\tau)$ (полная)	
	Church	Curry	Church	Curry
Type Checking	+	+	+	—
Type Inference	+	+	+	—
Type Inhabitation	+	+	—	—

* Сложность inhabitation достаточно высокая (известны PSPACE/EXPTIME-полные результаты).

Обозначения:

- + — задача разрешима (существует алгоритм)
- — — задача неразрешима

Ключевые выводы:

- **Church** — аннотации в термах; checking/inference механические
- **Curry** (полная System F) — нужно угадывать Λ и $[] \Rightarrow$ неразрешимо
- **Type inhabitation** для полной System F неразрешима (связь с логикой 2-го порядка)

Почему type inference неразрешим в Curry-стиле?

Проблема: без явных типовых аппликаций неясно, когда применять полиморфную функцию к типу.

Пример: рассмотрим терм

$$\lambda f. \lambda x. (f\ x)$$

Возможные типы:

- ❶ $(\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$ — мономорфный
- ❷ $\forall \alpha. \forall \beta. (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$ — полиморфный
- ❸ $(\forall \alpha. \alpha \rightarrow \alpha) \rightarrow \beta \rightarrow \beta$ — полиморфный аргумент
- ❹ Множество других вариантов...

Проблема: нет *главного* (principal) типа, который был бы самым общим.

Замечание: в упрощённой System F (prenex polymorphism) главный тип существует, но в полной — нет.

Type inhabitation: связь с логикой

Соответствие Карри-Говарда

Задача обитаемости типа — это задача **доказуемости формулы** в логике второго порядка.

- Тип $\leftrightarrow$ Формула
- Терм $\leftrightarrow$ Доказательство
- $\Gamma \vdash M : \sigma \leftrightarrow \langle M \text{ — доказательство формулы } \sigma \text{ из предположений } \Gamma \rangle$

Примеры:

- $\forall \alpha. \alpha \rightarrow \alpha$ — тавтология (закон тождества)
Обитаем термом $\lambda x. x$
- $\forall \alpha. \forall \beta. \alpha \rightarrow \beta \rightarrow \alpha$ — закон константы
Обитаем термом $\lambda x. \lambda y. x$
- $\forall \alpha. \alpha$ — ложь
Не обитаем (нет замкнутого терма этого типа)

Вывод: неразрешимость type inhabitation означает неразрешимость логики второго порядка. 39 / 76

План лекции

- 1 Введение в полиморфизм
- 2 Типовые переменные и контекст
- 3 Правила типизации System F
- 4 Стили типизации и разрешимость задач
- 5 Карри-стиль с явной полиморфной нотацией ($\lambda/[]$ как сахар)
- 6 Parametricity и «бесплатные теоремы»
- 7 Hindley-Milner vs System F: детальное сравнение
- 8 Дополнительные темы: кодирование и ограничения
- 9 Заключение

Типизация в стиле Карри: синтаксис

Термы без типовых аннотаций у переменных, но допускаем явные $\Lambda/[]$ для демонстрации специализаций.

Синтаксис термов с явной типовой абстракцией

$$M ::= x \mid \lambda x. M \mid M M \mid \Lambda \alpha. M \mid M [\tau]$$

где:

- $\Lambda \alpha. M$ — **типовая абстракция** (большая лямбда)
- $M [\tau]$ — **типовая аппликация** (применение к типу)

Примеры:

- $\Lambda \alpha. \lambda x. x$ — полиморфная функция тождества
- $(\Lambda \alpha. \lambda x. x) [\text{Nat}]$ — специализация к Nat
- $\Lambda \beta. \lambda x. \lambda y. y$ — полиморфная функция «второй аргумент»

Правила типизации в стиле Карри

Правило (Intro $\forall$)

$$\frac{\Gamma, \alpha : * \vdash M : \sigma \quad \alpha \notin \text{FV}(\Gamma)}{\Gamma \vdash \lambda\alpha. M : \forall\alpha. \sigma} \text{INTRO}\forall$$

Типовая абстракция $\lambda\alpha$ явно указывает обобщение по типу α .

Side condition: α не должна быть свободна в Γ .

Правило (Elim $\forall$)

$$\frac{\Gamma \vdash M : \forall\alpha. \sigma \quad \Gamma \vdash \rho : *}{\Gamma \vdash M[\rho] : \sigma[\alpha := \rho]} \text{ELIM}\forall$$

Типовая аппликация $M[\rho]$ явно указывает, к какому типу применяем.

Отличие от стиля Чёрча: в термовой абстракции $\lambda x. M$ тип x не указан.

Пример: типизация функции «второй проекции»

Терм: $\Lambda\beta. \lambda x. \lambda y. y$ (возвращает **второй** аргумент, игнорируя первый)

Контекст: $\Gamma = \{\alpha : *, x : \alpha\}$

Вывод типа:

$$\frac{\frac{\frac{\frac{y : \beta \in \Gamma}{\{\beta : *, \alpha : *, x : \alpha, y : \beta\} \vdash y : \beta}}{\{\beta : *, \alpha : *, x : \alpha\} \vdash \lambda y. y : \beta \rightarrow \beta}}{\{\beta : *, \alpha : *\} \vdash \lambda x. \lambda y. y : \alpha \rightarrow \beta \rightarrow \beta}}{\{\beta : *\} \vdash \lambda x. \lambda y. y : \forall \alpha. \alpha \rightarrow \beta \rightarrow \beta}}{\emptyset \vdash \Lambda\beta. \lambda x. \lambda y. y : \forall \beta. \forall \alpha. \alpha \rightarrow \beta \rightarrow \beta}$$

Результат: $\Lambda\beta. \lambda x. \lambda y. y : \forall \beta. \forall \alpha. \alpha \rightarrow \beta \rightarrow \beta$

Применение полиморфной функции

Пример: применим функцию «второй проекции» к конкретным типам

Пусть $F = \lambda\beta. \lambda x. \lambda y. y : \forall\beta. \forall\alpha. \alpha \rightarrow \beta \rightarrow \beta$ (возвращает **второй** аргумент)

Применение к типу Nat:

$$F \text{ Nat} : \forall\alpha. \alpha \rightarrow \text{Nat} \rightarrow \text{Nat}$$

Применение ко второму типу Bool:

$$(F \text{ Nat}) \text{ Bool} : \text{Bool} \rightarrow \text{Nat} \rightarrow \text{Nat}$$

Применение к термам:

$$((F \text{ Nat}) \text{ Bool}) \text{ true } 42 : \text{Nat}$$

результат: 42 (возвращает второй аргумент)

Сравнение стилей: итоговая таблица

Аспект	Стиль Чёрча	Стиль Карри
Термовая абстракция	$\lambda x^\sigma. M$	$\lambda x. M$
Типовая абстракция	$\Lambda \alpha. M$	неявная
Типовая аппликация	$M[\tau]$	неявная
Type checking	разрешим	зависит
Type inference	тривиален	сложен/неразрешим
Практичность	более явный	более компактный
Примеры языков	F*, Coq, Agda	ML, Haskell

Вывод:

- **Чёрч**: больше контроля, легче проверять
- **Карри**: удобнее писать, но сложнее выводиться типы

Замечание: современные языки часто используют гибрид — частичный вывод типов с возможностью явных аннотаций.

План лекции

- 1 Введение в полиморфизм
- 2 Типовые переменные и контекст
- 3 Правила типизации System F
- 4 Стили типизации и разрешимость задач
- 5 Карри-стиль с явной полиморфной нотацией ($\lambda/[]$ как сахар)
- 6 Parametricity и «бесплатные теоремы»**
- 7 Hindley-Milner vs System F: детальное сравнение
- 8 Дополнительные темы: кодирование и ограничения
- 9 Заключение

Что такое parametricity?

Интуиция

Из типа $\forall \alpha. \alpha \rightarrow \alpha$ следует, что функция не может «заглянуть» внутрь α .

Следовательно, единственная тотальная реализация — это $\lambda x. x$ (функция тождества).

Формальное утверждение (Reynolds, 1983):

Полиморфная функция ведёт себя *одинаково* для всех типов — её поведение определяется только структурой типа, а не конкретным типом.

Следствия:

- $f : \forall \alpha. \alpha \rightarrow \alpha \Rightarrow f \equiv \lambda x. x$ (единственность)
- $g : \forall \alpha \beta. (\alpha \rightarrow \beta) \rightarrow \text{List } \alpha \rightarrow \text{List } \beta$
 $\Rightarrow g$ сохраняет структуру списка
- Полиморфные функции не могут использовать runtime type information

Free theorems (Wadler, 1989)

Идея: из полиморфного типа автоматически следуют теоремы о поведении функции.

Пример 1: $\text{reverse} : \forall \alpha. \text{List } \alpha \rightarrow \text{List } \alpha$

Free theorem для reverse

Для любой функции $f : \alpha \rightarrow \beta$:

$$\text{map } f \circ \text{reverse} = \text{reverse} \circ \text{map } f$$

Интуиция: `reverse` не может «заглянуть» внутрь элементов (параметричность), поэтому порядок применения `map` и `reverse` не важен.

Пример 2: $\text{head} : \forall \alpha. \text{List } \alpha \rightarrow \alpha$

Free theorem для head

Для любой функции $f : \alpha \rightarrow \beta$ и списка xs :

$$f (\text{head } xs) = \text{head } (\text{map } f \ xs)$$

Доказательство «бесплатной теоремы» (идейно)

Для: $\text{reverse} : \forall \alpha. \text{List } \alpha \rightarrow \text{List } \alpha$

Хотим доказать: $\text{map } f \circ \text{reverse} = \text{reverse} \circ \text{map } f$

Идея (Wadler):

- 1 Из полиморфности reverse следует, что для любого отношения $R \subseteq \alpha \times \beta$:

$$R^* \Rightarrow \text{List } R$$

где $\text{List } R$ — покомпонентное применение R к элементам списка.

- 2 Выбираем $R = \{(x, f(x)) \mid x : \alpha\}$ (граф функции f).

- 3 Тогда $\text{List } R$ — это $(xs, \text{map } f \ xs)$.

- 4 Применяя reverse к обеим сторонам:

$$(\text{reverse } xs, \text{reverse } (\text{map } f \ xs)) \in \text{List } R$$

- 5 Но это эквивалентно:

$$\text{reverse } (\text{map } f \ xs) = \text{map } f \ (\text{reverse } xs)$$

Когда parametricity нарушается

Free theorems справедливы только при соблюдении **параметричности**.

Нарушители параметричности:

В Haskell:

- `unsafeCoerce` — принудительное приведение типов
- `undefined`, `error` — дивергенция
- `seq` — строгость и порядок вычислений
- FFI (Foreign Function Interface)

В других языках:

- `typeof/reflection` (Java, C#)
- Runtime type checks
- Downcasting (OOP)
- `instanceof` (Java)

Пример нарушения

```
{-# LANGUAGE TypeApplications #-}
import Type.Reflection (Typeable, typeRep, eqTypeRep, (:~~:)(...))

evil :: forall a. Typeable a => a -> String
evil _ = case eqTypeRep (typeRep @a) (typeRep @Int) of
```

Применения free theorems:

1 Оптимизации компилятора:

- Fusion laws: $\text{map } g \circ \text{map } f = \text{map } (g \circ f)$
(следует из free theorem для map)
- Short-cut fusion в GHC Haskell

2 Рефакторинг кода:

- Безопасные преобразования кода без изменения семантики
- Equational reasoning

3 Верификация программ:

- Автоматическое доказательство свойств из типов
- Property-based testing (QuickCheck)

4 Понимание API:

- Тип говорит, что функция *не может* делать
- «If it compiles, it works» (в определённом смысле)

План лекции

- 1 Введение в полиморфизм
- 2 Типовые переменные и контекст
- 3 Правила типизации System F
- 4 Стили типизации и разрешимость задач
- 5 Карри-стиль с явной полиморфной нотацией ($\lambda/[]$ как сахар)
- 6 Parametricity и «бесплатные теоремы»
- 7 Hindley-Milner vs System F: детальное сравнение**
- 8 Дополнительные темы: кодирование и ограничения
- 9 Заключение

Что такое Hindley-Milner?

Определение

Hindley-Milner (HM) — это система типов с автоматическим выводом типов, основанная на:

- **Let-полиморфизме** — обобщение типов только в `let`-связываниях
- **Rank-1 полиморфизме** — кванторы $\forall$ только снаружи (prenex form)
- **Алгоритме W** (Damas-Milner, 1982) — полный и практически эффективный вывод типов

Ключевая особенность: программист *не пишет* типовые аннотации — система сама выводит **principal type** (самый общий тип).

Где используется:

- **ML-семейство:** OCaml, Standard ML, F#
- **Haskell** — HM + type classes + расширения (RankNTypes, GADTs, ...)
- **Elm, PureScript, ReasonML** — современные функциональные языки

Почему важно: баланс между выразительностью и автоматизацией — «пишешь как динамический язык, работает как типизированный».

Таблица сравнения HM и System F

Свойство	HM (ML/Haskell)	System F (λ_2)
Полиморфизм	let-полиморфизм	полный $\forall$
Ранг типов	rank-1 (prenex)	произвольный
Вывод типов	алгоритм W (разрешим)	неразрешим
Проверка типов	разрешима	разрешима
Аннотации	минимальны	все $\Lambda/[]$ явно
Impredicativity	нет	да
Инстанцирование	неявное	явное $[\tau]$
Principal type	существует	нет (в общем)
Type classes	да (ad-hoc)	кодируется
Практичность	высокая	ниже (много аннотаций)

Ключевое различие: HM жертвует выразительностью ради автоматического вывода типов.

Алгоритм W — структурный обход AST с унификацией; практически эффективен, но в худшем случае размер главного типа (и время вывода) может быть экспоненциальным.

Let-полиморфизм через схемы типов

Пример (Haskell/ML):

```
let id = \x -> x in (id 5, id True)
```

Вывод типов в HM:

❶ **Обобщение** (generalization):

$\text{id} :: \forall \alpha. \alpha \rightarrow \alpha$ (тип-схема)

❷ **Инстанцирование** (неявно при каждом использовании):

- Первое использование: $\text{id}[\text{Int}] : \text{Int} \rightarrow \text{Int}$
- Второе использование: $\text{id}[\text{Bool}] : \text{Bool} \rightarrow \text{Bool}$

В System F (явно):

```
let id = Lambda alpha. \x:alpha. x  
in (id[Nat] 5, id[Bool] True)
```

В HM: инстанцирование автоматическое, но только в let-связывании.

Алгоритм W как катаморфизм

Связь с лекцией 5: алгоритм W — это катаморфизм на синтаксическом дереве.

Структурная рекурсия

Алгоритм W обходит AST терма, применяя правила типизации к каждому конструктору:

$$\text{infer} : \text{Term} \rightarrow \text{State Subst (Maybe Type)}$$

Это катаморфизм с алгеброй $\alpha : \text{TermF}(A) \rightarrow A$:

- $\text{Var}(x) \mapsto$ поиск в контексте $\Gamma(x)$
- $\text{Lam}(x, M) \mapsto$ введение новой переменной + рекурсия + построение $\tau \rightarrow \sigma$
- $\text{App}(M, N) \mapsto$ рекурсия + унификация типов
- $\text{Let}(x, M, N) \mapsto$ рекурсия + **обобщение** (generalization) + рекурсия

Ключевое отличие от простого fold: алгоритм W — это катаморфизм *с состоянием* (подстановки типовых переменных).

Универсальное свойство и корректность алгоритма W

Связь с F-алгебрами (из лекции 4):

Раз алгоритм W — катаморфизм, он удовлетворяет уравнению:

$$\text{infer} \circ \text{in} = \alpha \circ F(\text{infer})$$

Следствие: композициональность

Тип составного терма определяется только типами его подтермов и правилом комбинирования.

Это гарантирует:

- **Корректность:** вывод типа не зависит от порядка обхода
- **Единственность:** principal type определяется однозначно
- **Модульность:** можно выводиться типы для подвыражений независимо

Практическое значение:

- Алгоритмы на AST (компиляция, оптимизация) часто являются катаморфизмами
- Fusion laws позволяют оптимизировать композиции проходов компилятора

Ограничение HM: только rank-1

Что невозможно в HM без расширений:

```
-- HM (rank-1), without annotation:  
applyToId f = (f 42, f True)  
  
-- Error:  
-- Couldn't match type 'Int' with 'Bool'  
-- Expected type: Bool -> Bool  
--   Actual type: Int  -> Int
```

Почему?

В HM (rank-1) квантор $\forall$ может быть только *снаружи* типа функции.

Тип $(\forall \alpha. \alpha \rightarrow \alpha) \rightarrow \dots$ — это rank-2, запрещён в HM.

Решение: расширение `{-# LANGUAGE RankNTypes #-}` позволяет написать:

```
applyToId :: (forall a. a -> a) -> (Int, Bool)  
applyToId f = (f 42, f True)
```

Пример: типизуемо в System F, но не в HM

System F (rank-2 type):

$$\text{idid} : \forall \alpha. (\forall \beta. \beta \rightarrow \beta) \rightarrow (\alpha \rightarrow \alpha)$$

$$\text{idid} = \Lambda \alpha. \lambda f. (\forall \beta. \beta \rightarrow \beta). f[\alpha]$$

Этот тип **невозможен** в обычном HM (только rank-1).

Интуиция различия:

- **HM**: обобщение только в let-связывании
(prenex polymorphism: кванторы только снаружи)
- **System F**: обобщение везде, где угодно
(impredicative: кванторы в любом месте типа)

Ещё пример (rank-3):

$$(\forall \alpha. \alpha \rightarrow \alpha) \rightarrow ((\forall \beta. \beta \rightarrow \beta) \rightarrow \text{Int}) \rightarrow \text{Int}$$

В HM: нужно расширение RankNTypes или ImpredicativeTypes.

Почему HM разрешим, а System F — нет?

Причины разрешимости вывода типов в HM:

1 Prenex polymorphism (rank-1):

Кванторы только снаружи $\rightarrow$ типы проще, нет вложенности.

2 Let-полиморфизм:

Обобщение только в `let` $\rightarrow$ локализация инстанцирования.

3 Principal type property:

Для каждого терма существует *самый общий* тип (principal type).
Алгоритм W находит его за полиномиальное время.

4 Синтаксически направленный вывод:

Унификация типовых переменных детерминирована структурой терма.

В System F:

- Нет principal type (множество несравнимых типизаций)
- Нужно угадывать, куда вставить Λ и где применить $[\tau]$
- Пространство поиска экспоненциально/бесконечно

Пример: отсутствие principal type в System F

Рассмотрим терм (в стиле Карри):

$$\lambda f. \lambda x. (f\ x)$$

Возможные типы в System F:

- ❶ $(\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$ (мономорфный)
- ❷ $\forall \alpha \beta. (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$ (полностью полиморфный)
- ❸ $\forall \beta. (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$ (α фиксирован в контексте)
- ❹ $(\forall \alpha. \alpha \rightarrow \alpha) \rightarrow \beta \rightarrow \beta$ (пoлиморфный аргумент, rank-2)
- ❺ $\forall \beta. (\forall \alpha. \alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$ (частично полиморфный, некорректен)
- ❻ ... и множество других вариантов

Проблема: нет *самого общего* типа, который обобщал бы все остальные. Типы (1), (2), (4) — несравнимы по общности.

В НМ: principal type — это (2), все остальные — его инстанции.

План лекции

- 1 Введение в полиморфизм
- 2 Типовые переменные и контекст
- 3 Правила типизации System F
- 4 Стили типизации и разрешимость задач
- 5 Карри-стиль с явной полиморфной нотацией ($\lambda/[]$ как сахар)
- 6 Parametricity и «бесплатные теоремы»
- 7 Hindley-Milner vs System F: детальное сравнение
- 8 Дополнительные темы: кодирование и ограничения**
- 9 Заключение

Church-кодировки в System F

Идея: типы данных можно кодировать через полиморфные функции.

Булевы значения

$$\text{Bool} \equiv \forall \alpha. \alpha \rightarrow \alpha \rightarrow \alpha$$

Интуиция: «функция выбора» — возвращает первый или второй аргумент.

Кодировки:

- $\text{true} := \Lambda \alpha. \lambda x^\alpha. \lambda y^\alpha. x$ (выбирает первый)
- $\text{false} := \Lambda \alpha. \lambda x^\alpha. \lambda y^\alpha. y$ (выбирает второй)

Проверка типа:

$$\text{true}[\text{Nat}] \ 42 \ 0 \equiv (\lambda x. \lambda y. x) \ 42 \ 0 \mapsto 42$$

Church-кодировки: пары и списки

Пары (произведение типов)

$$\alpha \times \beta \equiv \forall \gamma. (\alpha \rightarrow \beta \rightarrow \gamma) \rightarrow \gamma$$

Конструктор:

$$\text{pair} := \Lambda \alpha. \Lambda \beta. \lambda x^\alpha. \lambda y^\beta. \Lambda \gamma. \lambda f^{\alpha \rightarrow \beta \rightarrow \gamma}. f \ x \ y$$

Проекции:

$$\text{fst} := \Lambda \alpha. \Lambda \beta. \lambda p^{\alpha \times \beta}. p[\alpha] (\lambda x. \lambda y. x)$$

$$\text{snd} := \Lambda \alpha. \Lambda \beta. \lambda p^{\alpha \times \beta}. p[\beta] (\lambda x. \lambda y. y)$$

Ключевое свойство: тип γ — **impredicative** (может быть инстанцирован как угодно, в том числе самим полиморфным типом).

Важно: в НМ такие кодировки ломаются именно из-за predicative-ограничений — квантор $\forall$ не может быть аргументом, только снаружи.

Списки через fold

$$\text{List } \alpha \equiv \forall \beta. (\alpha \rightarrow \beta \rightarrow \beta) \rightarrow \beta \rightarrow \beta$$

Интуиция: список — это его свёртка (fold).

Конструкторы:

- $\text{nil} := \Lambda \alpha. \Lambda \beta. \lambda f^{\alpha \rightarrow \beta \rightarrow \beta}. \lambda z^{\beta}. z$
- $\text{cons} := \Lambda \alpha. \lambda x^{\alpha}. \lambda xs^{\text{List } \alpha}. \Lambda \beta. \lambda f. \lambda z. f \ x \ (xs[\beta] \ f \ z)$

Пример: список [1, 2, 3]

$\text{cons } 1 \ (\text{cons } 2 \ (\text{cons } 3 \ \text{nil}))$

Вывод: System F достаточно мощна, чтобы кодировать произвольные алгебраические типы данных.

Impredicativity: почему это важно

Impredicative polymorphism — квантор $\forall$ может быть инстанцирован самим полиморфным типом.

Пример impredicativity

Пусть $p : \forall \gamma. (\alpha \rightarrow \beta \rightarrow \gamma) \rightarrow \gamma$ (пара).

Мы можем применить её к типу $\gamma := \forall \delta. \delta \rightarrow \delta$:

$$p[\forall \delta. \delta \rightarrow \delta] (\lambda x. \lambda y. \dots)$$

Тип $\forall \delta. \delta \rightarrow \delta$ — сам полиморфный, но допустим как инстанция γ .

Контраст с HM:

- В HM полиморфизм **predicative** — квантор всегда «снаружи»
- В System F полиморфизм **impredicative** — квантор может быть где угодно

Следствие: Church-кодировки требуют impredicativity — без неё не закодировать даже пары.

Экзистенциальные типы: мотивация

Проблема: как скрыть внутреннее представление типа данных?

Пример (абстрактный счётчик):

- Мы хотим предоставить операции `new`, `inc`, `get`
- Но скрыть, что счётчик реализован как `Nat`

Экзистенциальный тип

$$\text{Counter} := \exists \alpha. \{ \text{new} : \alpha, \text{inc} : \alpha \rightarrow \alpha, \text{get} : \alpha \rightarrow \text{Nat} \}$$

Интуиция: «существует некий тип α (скрытый), вместе с операциями над ним».

Ключевое свойство: клиент *не знает*, что $\alpha = \text{Nat}$ — он видит только интерфейс.

Вопрос: как кодировать $\exists$ в System F?

Экзистенциальные типы: кодировка в System F

Church-encoding для $\exists$

$$\exists \alpha. \tau \quad \equiv \quad \forall \beta. (\forall \alpha. \tau \rightarrow \beta) \rightarrow \beta$$

Интуиция: «чтобы получить β из $\exists \alpha. \tau$, нужно предоставить continuation, которая работает для любого α ».

Операции:

- **Pack** (создание экзистенциала):

$$\text{pack} [\sigma, M] : \exists \alpha. \tau \quad \text{если} \quad M : \tau[\alpha := \sigma]$$

$$\text{pack} [\sigma, M] := \Lambda \beta. \lambda f^{\forall \alpha. \tau \rightarrow \beta}. f[\sigma] M$$

- **Unpack** (распаковка):

$$\text{unpack} \{ \alpha, x \} = E_1 \text{ in } E_2 : \beta$$

$$E_1[\beta] (\Lambda \alpha. \lambda x. E_2)$$

Важно: β — свежий тип в области E_2 , чтобы избежать коллизий с существующими типовыми

Экзистенциальные типы: связь с практикой

Где встречаются $\exists$ -типы?

Rust: trait objects

```
trait Draw {  
    fn draw(&self);  
}  
  
// dyn Draw ~ exists T. (T, vtable)  
fn render(obj: &dyn Draw) {  
    obj.draw();  
}
```

Тип: $\text{dyn Draw} \approx \exists \alpha. (\alpha, \alpha \rightarrow \text{Unit})$

Вывод: экзистенциальные типы — это формальная основа для абстракции данных и ООП.

C++: виртуальные функции

```
class Shape {  
    virtual void draw() = 0;  
};  
  
// Shape* ~ exists T. (T*, vtable*)  
void render(Shape* s) {  
    s->draw();  
}
```

Тип: указатель на vtable скрывает конкретный тип объекта.

Value restriction в HM: проблема

Наивный let-полиморфизм в присутствии мутабельности **нарушает type safety**.

Проблемный пример (ML/OCaml)

```
let id = ( $\lambda x. x$ ) ( $\lambda y. y$ )  
in ...
```

Наивный алгоритм W выводит: $\text{id} : \forall \alpha. \alpha \rightarrow \alpha$

Но если добавить мутабельность (например, ссылки):

```
let r = ref ( $\lambda x. x$ )  
in ( $r := (\lambda n. n + 1)$ ); (!r) true
```

Наивный вывод: $r : \forall \alpha. \text{ref } (\alpha \rightarrow \alpha)$

Проблема: присваиваем $\text{Nat} \rightarrow \text{Nat}$, читаем как $\text{Bool} \rightarrow \text{Bool} \rightarrow \text{type safety}$ нарушена.

Value restriction: решение

Правило value restriction (Wright, 1995)

Обобщать типовые переменные в let-связывании можно только если правая часть — **синтаксическое значение** (value):

Значения:

- Переменные: x
- Абстракции: $\lambda x. M$
- Конструкторы данных: $\text{Cons } x \ xs$
- Константы: $42, \text{true}$

НЕ значения:

- Применения функций: $f \ x$
- Операции: $x + y$
- Разыменование ссылок: $!r$

Value restriction: последствия

Плюсы:

- Сохраняет type safety в присутствии мутабельности
- Простое синтаксическое правило
- Работает для OCaml, SML, F#

Минусы:

- Иногда отвергает корректные программы:

`let id = ( $\lambda x. x$ ) ( $\lambda y. y$ ) in ...`

Здесь `id` получает мономорфный тип, хотя программа безопасна.

- Программисту приходится переписывать:

`let id =  $\lambda y. y$  in ...` (это value)

Альтернатива: использовать линейные типы или region-based memory management (Rust borrow checker).

Haskell: нет value restriction, т.к. нет мутабельности (кроме IO monad).

План лекции

- 1 Введение в полиморфизм
- 2 Типовые переменные и контекст
- 3 Правила типизации System F
- 4 Стили типизации и разрешимость задач
- 5 Карри-стиль с явной полиморфной нотацией ($\lambda/[]$ как сахар)
- 6 Parametricity и «бесплатные теоремы»
- 7 Hindley-Milner vs System F: детальное сравнение
- 8 Дополнительные темы: кодирование и ограничения
- 9 **Заключение**

Что мы изучили:

- 1 **System F** (λ_2) — полиморфизм второго порядка с квантором $\forall$
- 2 **Правила F/I/E/C для $\forall$** — Formation, Introduction ($\Lambda\alpha$), Elimination [τ], Computation (β -редукция)
- 3 **Два стиля типизации** — Чёрч (явный) vs Карри (неявный) и их влияние на разрешимость
- 4 **Parametricity и free theorems** — типы гарантируют свойства функций
- 5 **HM vs System F** — let-порморфизм vs полный $\forall$, разрешимость vs выразительность
- 6 **Church-кодировки** — Bool, пары, списки через $\forall$ (impredicativity)
- 7 **Экзистенциальные типы** — $\exists$ через $\forall$, связь с vtables/trait objects
- 8 **Value restriction** — как HM сохраняет soundness при мутабельности

Ключевой инсайт: типы — это не просто аннотации, а *доказательства свойств программ*.

Типы и термы:

- $\forall\alpha. \sigma$ — полиморфный тип
- $\Lambda\alpha. M$ — типовая абстракция
- $M \tau$ — типовая аппликация
- $\alpha : *$ — типовая переменная

Контекст:

- $\Gamma = \{\alpha : *, x : \sigma\}$
- Упорядоченный список
- $FV(\sigma) \subseteq \Gamma$

Правила:

- $\text{Intro}\forall$ — обобщение
- $\text{Elim}\forall$ — специализация
- Сохранение типов
- Прогресс вычислений

Системы:

- $\lambda_2(\tau_0)$ — prenex
- $\lambda_2(\tau)$ — полная
- Чёрч vs Карри

Где используется System F:

- **ML-семейство языков** (OCaml, F#, SML) — параметрический полиморфизм
- **Haskell** — система типов основана на System F с расширениями
- **Rust** — generics компилируются через мономорфизацию
- **Java** — обобщения со стиранием типов (type erasure)
- **C#** — реифицированные обобщения (без стирания)
- **Scala** — higher-kinded types и полиморфизм
- **Proof assistants** (Coq, Lean) — System F как часть теории

Главное: полиморфизм позволяет писать переиспользуемый, типобезопасный код.